

Main

====

```
BEGIN PUBLIC CLASS Main

    DECLARE PRIVATE STATIC FINAL STRING CSV_FILE = "data.csv"

    PUBLIC STATIC METHOD main(args)
        CALL Logger.open()

        DECLARE SensorArray loadedReadings
        SET loadedReadings = CALL GreenhouseDataStorage.loadCSV(CSV_FILE)

        DECLARE Menu menu
        SET menu = NEW Menu(loadedReadings, CSV_FILE)

        CALL menu.run()

        CALL Logger.close()
    END METHOD

END CLASS
```

Menu

====

```
BEGIN PUBLIC CLASS Menu

    DECLARE PRIVATE BufferedReader consoleInputReader
    DECLARE PRIVATE SensorArray loadedReadings
    DECLARE PRIVATE STRING csvFilename

    CONSTRUCTOR Menu(SensorArray loadedReadings, STRING csvFilename)
        SET this.loadedReadings = loadedReadings
        SET this.csvFilename = csvFilename
        SET this.consoleInputReader = NEW BufferedReader(NEW InputStreamReader(System.in))
    END CONSTRUCTOR

    PUBLIC METHOD run()
        DECLARE BOOLEAN menuIsRunning = TRUE

        WHILE menuIsRunning EQUALS TRUE
            CALL showMainMenu()
            DECLARE INTEGER userChoice = CALL readIntegerInRange("choice: ", 1, 6)

            IF userChoice EQUALS 1 THEN
                CALL handleOverallStats()
            ELSE IF userChoice EQUALS 2 THEN
                CALL handleStatsByZone()
            ELSE IF userChoice EQUALS 3 THEN
                CALL handleStatsByType()
            ELSE IF userChoice EQUALS 4 THEN
                CALL handleAddReading()
            ELSE IF userChoice EQUALS 5 THEN
                CALL handleDeleteReading()
            ELSE IF userChoice EQUALS 6 THEN
                SET menuIsRunning = FALSE
            END IF
        END WHILE
    END METHOD

END CLASS
```

```

    END WHILE
END METHOD

```

```

PRIVATE METHOD showMainMenu()
    CALL Logger.logBlank()
    CALL Logger.log("Main menu")
    CALL Logger.log("1. overall statistics")
    CALL Logger.log("2. stats by zone")
    CALL Logger.log("3. stats by sensor type")
    CALL Logger.log("4. add a reading")
    CALL Logger.log("5. delete a reading")
    CALL Logger.log("6. exit")
END METHOD

```

```

PRIVATE METHOD handleOverallStats()
    CALL showStatsSubMenu(loadedReadings, "entire greenhouse")
END METHOD

```

```

PRIVATE METHOD handleStatsByZone()
    DECLARE STRING selectedZone = CALL readValidZone()
    DECLARE SensorArray filteredReadings = CALL
loadedReadings.filterByZone(selectedZone)
    CALL showStatsSubMenu(filteredReadings, "zone " + selectedZone)
END METHOD

```

```

PRIVATE METHOD readValidZone() RETURNS STRING
    DECLARE STRING ARRAY zones = {"ZoneA", "ZoneB", "ZoneC"}

    CALL Logger.logBlank()
    CALL Logger.log("Select zone")

    FOR i = 0 TO zones.length - 1
        CALL Logger.log((i + 1) + ". " + zones[i])
    END FOR

    DECLARE INTEGER userSelection = CALL readIntegerInRange("zone: ", 1, zones.length)
    RETURN zones[userSelection - 1]
END METHOD

```

```

PRIVATE METHOD handleStatsByType()
    DECLARE STRING selectedSensorType = CALL readValidSensorType()
    DECLARE SensorArray filteredReadings = CALL
loadedReadings.filterBySensorType(selectedSensorType)
    CALL showStatsSubMenu(filteredReadings, "type " + selectedSensorType)
END METHOD

```

```

PRIVATE METHOD readValidSensorType() RETURNS STRING
    DECLARE STRING ARRAY sensorTypes = {"temperature", "humidity", "soilMoisture",
"light"}

    CALL Logger.logBlank()
    CALL Logger.log("Select sensor type")

    FOR i = 0 TO sensorTypes.length - 1
        CALL Logger.log((i + 1) + ". " + sensorTypes[i])
    END FOR

```

```

    END FOR

    DECLARE INTEGER userSelection = CALL readIntegerInRange("type: ", 1,
sensorTypes.length)
    RETURN sensorTypes[userSelection - 1]
END METHOD

PRIVATE METHOD handleAddReading()
    CALL Logger.logBlank()
    CALL Logger.log("Add new reading")

    DECLARE STRING sensorID = CALL readNonEmptyString("sensor ID: ")

    WHILE loadedReadings.hasSensorID(sensorID) EQUALS TRUE
        CALL Logger.log("That sensor ID is already in the data.")
        SET sensorID = CALL readNonEmptyString("sensor ID: ")
    END WHILE

    DECLARE STRING sensorType = CALL readValidSensorType()
    DECLARE STRING zone = CALL readValidZone()
    DECLARE DOUBLE sensorValue = CALL readDoubleNumber("value: ")

    DECLARE INTEGER day = CALL readIntegerInRange("day (1-31): ", 1, 31)
    DECLARE INTEGER month = CALL readIntegerInRange("month (1-12): ", 1, 12)
    DECLARE INTEGER year = CALL readIntegerInRange("year (2000-2100): ", 2000, 2100)
    DECLARE INTEGER hour = CALL readIntegerInRange("hour (0-23): ", 0, 23)
    DECLARE INTEGER minute = CALL readIntegerInRange("minute (0-59): ", 0, 59)

    DECLARE Timestamp newReadingTimestamp = NEW Timestamp(day, month, year, hour,
minute)
    DECLARE SensorReading newReading = NEW SensorReading(sensorID, sensorType, zone,
sensorValue, newReadingTimestamp)

    CALL loadedReadings.add(newReading)
    CALL Logger.log("Added: " + newReading)
    CALL GreenhouseDataStorage.saveCSV(csvFilename, loadedReadings)
END METHOD

PRIVATE METHOD handleDeleteReading()
    IF loadedReadings.getCount() EQUALS 0 THEN
        CALL Logger.log("No readings to delete.")
    ELSE
        CALL Logger.logBlank()
        CALL Logger.log("Current readings")

        FOR i = 0 TO loadedReadings.getCount() - 1
            CALL Logger.log((i + 1) + ". " + loadedReadings.get(i))
        END FOR

        DECLARE INTEGER userSelection = CALL readIntegerInRange("delete number: ", 1,
loadedReadings.getCount())
        DECLARE SensorReading removedReading = CALL loadedReadings.get(userSelection -
1)

        CALL loadedReadings.deleteAt(userSelection - 1)
        CALL Logger.log("Deleted: " + removedReading)
        CALL GreenhouseDataStorage.saveCSV(csvFilename, loadedReadings)
    END IF
END METHOD

```

```

PRIVATE METHOD showStatsSubMenu(SensorArray filteredReadings, STRING contextLabel)
    CALL Logger.logBlank()
    CALL Logger.log("Stats for " + contextLabel)
    CALL Logger.log("1. total readings")
    CALL Logger.log("2. average value")
    CALL Logger.log("3. minimum value")
    CALL Logger.log("4. maximum value")
    CALL Logger.log("5. number outside safe range")
    CALL Logger.log("6. percent outside safe range")
    CALL Logger.log("7. all of the above")

    DECLARE INTEGER userChoice = CALL readIntegerInRange("stat: ", 1, 7)
    CALL Logger.logBlank()

    IF userChoice EQUALS 1 THEN
        CALL Logger.log("Total: " + Math.totalReadings(filteredReadings))
    ELSE IF userChoice EQUALS 2 THEN
        CALL Logger.log("Average: " + Math.average(filteredReadings))
    ELSE IF userChoice EQUALS 3 THEN
        CALL Logger.log("Minimum: " + Math.minimum(filteredReadings))
    ELSE IF userChoice EQUALS 4 THEN
        CALL Logger.log("Maximum: " + Math.maximum(filteredReadings))
    ELSE IF userChoice EQUALS 5 THEN
        CALL Logger.log("Number outside safe range: " +
Math.outOfRangeCount(filteredReadings))
    ELSE IF userChoice EQUALS 6 THEN
        CALL Logger.log("Percent outside safe range: " +
Math.outOfRangePercent(filteredReadings))
    ELSE IF userChoice EQUALS 7 THEN
        CALL Logger.log("Total: " + Math.totalReadings(filteredReadings))
        CALL Logger.log("Average: " + Math.average(filteredReadings))
        CALL Logger.log("Minimum: " + Math.minimum(filteredReadings))
        CALL Logger.log("Maximum: " + Math.maximum(filteredReadings))
        CALL Logger.log("Number outside safe range: " +
Math.outOfRangeCount(filteredReadings))
        CALL Logger.log("Percent outside safe range: " +
Math.outOfRangePercent(filteredReadings))
    END IF
END METHOD

PRIVATE METHOD readLineFromConsole(STRING promptToShow) RETURNS STRING
    PRINT promptToShow WITHOUT NEW LINE

    TRY
        DECLARE STRING rawInputLine = READ LINE FROM consoleInputReader
        IF rawInputLine EQUALS NULL THEN
            RETURN ""
        ELSE
            RETURN rawInputLine.trim()
        END IF
    CATCH IOException
        RETURN ""
    END TRY
END METHOD

PRIVATE METHOD readIntegerInRange(STRING promptToShow, INTEGER minimumAllowedValue,
INTEGER maximumAllowedValue) RETURNS INTEGER

```

```
    DECLARE INTEGER validatedInteger = 0
    DECLARE BOOLEAN inputAccepted = FALSE

    WHILE inputAccepted EQUALS FALSE
        DECLARE STRING userInputString = CALL readLineFromConsole(promptToShow)

        IF Validator.isInteger(userInputString) EQUALS TRUE THEN
            DECLARE INTEGER parsedInteger = PARSE userInputString AS INTEGER

            IF parsedInteger >= minimumAllowedValue AND parsedInteger <=
maximumAllowedValue THEN
                SET validatedInteger = parsedInteger
                SET inputAccepted = TRUE
            ELSE
                CALL Logger.log("Enter a number between " + minimumAllowedValue + " and
" + maximumAllowedValue + ".")
            END IF
        ELSE
            CALL Logger.log("Enter a number between " + minimumAllowedValue + " and " +
maximumAllowedValue + ".")
        END IF
    END WHILE

    RETURN validatedInteger
END METHOD

PRIVATE METHOD readDoubleNumber(STRING promptToShow) RETURNS DOUBLE
    DECLARE DOUBLE validatedDouble = 0.0
    DECLARE BOOLEAN inputAccepted = FALSE

    WHILE inputAccepted EQUALS FALSE
        DECLARE STRING userInputString = CALL readLineFromConsole(promptToShow)

        IF Validator.isNumeric(userInputString) EQUALS TRUE THEN
            SET validatedDouble = PARSE userInputString AS DOUBLE
            SET inputAccepted = TRUE
        ELSE
            CALL Logger.log("Enter a number.")
        END IF
    END WHILE

    RETURN validatedDouble
END METHOD

PRIVATE METHOD readNonEmptyString(STRING promptToShow) RETURNS STRING
    DECLARE STRING validatedString = ""
    DECLARE BOOLEAN inputAccepted = FALSE

    WHILE inputAccepted EQUALS FALSE
        DECLARE STRING userInputString = CALL readLineFromConsole(promptToShow)

        IF userInputString.length() > 0 THEN
            SET validatedString = userInputString
            SET inputAccepted = TRUE
        ELSE
            CALL Logger.log("This cannot be empty.")
        END IF
    END WHILE
```

```

    RETURN validatedString
END METHOD

```

```

END CLASS

```

```

GreenhouseDataStorage
=====

```

```

BEGIN PUBLIC CLASS GreenhouseDataStorage

```

```

    PUBLIC STATIC METHOD loadCSV(String csvFilename) RETURNS SensorArray
        DECLARE SensorArray loadedReadings = NEW SensorArray()
        DECLARE BufferedReader csvReader = NULL

```

```

    TRY

```

```

        SET csvReader = OPEN csvFilename FOR READING
        READ AND IGNORE FIRST LINE

```

```

        DECLARE INTEGER currentLineNumber = 1
        DECLARE STRING currentLine = READ LINE FROM csvReader

```

```

        WHILE currentLine IS NOT NULL

```

```

            SET currentLineNumber = currentLineNumber + 1
            SET currentLine = currentLine.trim()

```

```

            IF currentLine.length() > 0 THEN

```

```

                DECLARE SensorReading parsedReading = CALL parseLine(currentLine)

```

```

                IF parsedReading IS NOT NULL THEN

```

```

                    CALL loadedReadings.add(parsedReading)

```

```

                ELSE

```

```

                    PRINT "Warning: skipped invalid row " + currentLineNumber

```

```

                END IF

```

```

            END IF

```

```

            SET currentLine = READ LINE FROM csvReader

```

```

        END WHILE

```

```

    CATCH IOException

```

```

        PRINT "Error loading file"

```

```

    END TRY

```

```

    IF csvReader IS NOT NULL THEN

```

```

        CLOSE csvReader

```

```

    END IF

```

```

    RETURN loadedReadings

```

```

END METHOD

```

```

PUBLIC STATIC METHOD saveCSV(String csvFilename, SensorArray readingsToSave)
    DECLARE BufferedWriter csvWriter = NULL

```

```

    TRY

```

```

        SET csvWriter = OPEN csvFilename FOR WRITING IN OVERWRITE MODE
        WRITE "day,month,year,hour,minute,sensorID,sensorType,zone,value"
        WRITE NEW LINE

```

```

        FOR i = 0 TO readingsToSave.getCount() - 1

```

```

            DECLARE SensorReading currentReading = CALL readingsToSave.get(i)

```

```

    DECLARE Timestamp currentTimestamp = CALL currentReading.getTimestamp()

    DECLARE STRING csvLine =
        currentTimestamp.getDayOfMonth() + "," +
        currentTimestamp.getMonthOfYear() + "," +
        currentTimestamp.getYear() + "," +
        currentTimestamp.getHour() + "," +
        currentTimestamp.getMinute() + "," +
        currentReading.getSensorID() + "," +
        currentReading.getSensorType() + "," +
        currentReading.getZone() + "," +
        currentReading.getValue()

    WRITE csvLine
    WRITE NEW LINE
END FOR

CATCH IOException
    PRINT "Error saving file"
END TRY

IF csvWriter IS NOT NULL THEN
    CLOSE csvWriter
END IF
END METHOD

PRIVATE STATIC METHOD parseLine(STRING csvLine) RETURNS SensorReading
    DECLARE SensorReading parsedReading = NULL
    DECLARE STRING ARRAY csvFields = NULL

    IF csvLine IS NOT NULL THEN
        SET csvFields = SPLIT csvLine BY "," KEEPING EMPTY FIELDS
    END IF

    IF csvFields EXISTS AND csvFields.length EQUALS 9 THEN
        DECLARE STRING dayString = csvFields[0].trim()
        DECLARE STRING monthString = csvFields[1].trim()
        DECLARE STRING yearString = csvFields[2].trim()
        DECLARE STRING hourString = csvFields[3].trim()
        DECLARE STRING minuteString = csvFields[4].trim()
        DECLARE STRING sensorID = csvFields[5].trim()
        DECLARE STRING sensorType = csvFields[6].trim()
        DECLARE STRING zone = csvFields[7].trim()
        DECLARE STRING valueString = csvFields[8].trim()

        DECLARE BOOLEAN rowIsValid = TRUE

        IF Validator.isInteger(dayString) EQUALS FALSE THEN SET rowIsValid = FALSE END
    IF
        IF Validator.isInteger(monthString) EQUALS FALSE THEN SET rowIsValid = FALSE END
    IF
        IF Validator.isInteger(yearString) EQUALS FALSE THEN SET rowIsValid = FALSE END
    IF
        IF Validator.isInteger(hourString) EQUALS FALSE THEN SET rowIsValid = FALSE END
    IF
        IF Validator.isInteger(minuteString) EQUALS FALSE THEN SET rowIsValid = FALSE
    END IF

    IF rowIsValid EQUALS TRUE THEN
        DECLARE INTEGER day = PARSE dayString AS INTEGER

```

```

        DECLARE INTEGER month = PARSE monthString AS INTEGER
        DECLARE INTEGER year = PARSE yearString AS INTEGER
        DECLARE INTEGER hour = PARSE hourString AS INTEGER
        DECLARE INTEGER minute = PARSE minuteString AS INTEGER

        IF Validator.isValidDay(day) EQUALS FALSE THEN SET rowIsValid = FALSE END IF
        IF Validator.isValidMonth(month) EQUALS FALSE THEN SET rowIsValid = FALSE
END IF
        IF Validator.isValidYear(year) EQUALS FALSE THEN SET rowIsValid = FALSE END
IF
        IF Validator.isValidHour(hour) EQUALS FALSE THEN SET rowIsValid = FALSE END
IF
        IF Validator.isValidMinute(minute) EQUALS FALSE THEN SET rowIsValid = FALSE
END IF
        IF Validator.isValidSensorID(sensorID) EQUALS FALSE THEN SET rowIsValid =
FALSE END IF
        IF Validator.isValidSensorType(sensorType) EQUALS FALSE THEN SET rowIsValid
= FALSE END IF
        IF Validator.isValidZone(zone) EQUALS FALSE THEN SET rowIsValid = FALSE END
IF
        IF Validator.isNumeric(valueString) EQUALS FALSE THEN SET rowIsValid = FALSE
END IF

        IF rowIsValid EQUALS TRUE THEN
            DECLARE DOUBLE sensorValue = PARSE valueString AS DOUBLE
            DECLARE Timestamp readingTimestamp = NEW Timestamp(day, month, year,
hour, minute)
            SET parsedReading = NEW SensorReading(sensorID, sensorType, zone,
sensorValue, readingTimestamp)
        END IF
    END IF
END IF

    RETURN parsedReading
END METHOD

```

END CLASS

SensorArray
=====

BEGIN PUBLIC CLASS SensorArray

```

    DECLARE PRIVATE STATIC FINAL INTEGER INITIAL_CAPACITY = 100
    DECLARE PRIVATE STATIC FINAL INTEGER CAPACITY_GROWTH_FACTOR = 2

```

```

    DECLARE PRIVATE ARRAY OF SensorReading readings
    DECLARE PRIVATE INTEGER numberOfReadings

```

```

    CONSTRUCTOR SensorArray()
        SET readings = NEW SensorReading ARRAY WITH SIZE INITIAL_CAPACITY
        SET numberOfReadings = 0
    END CONSTRUCTOR

```

```

    PUBLIC METHOD add(SensorReading newReading)
        IF numberOfReadings EQUALS readings.length THEN
            CALL growArray()
        END IF
    END METHOD

```



```
    SET readings[numberOfReadings] = newReading
    SET numberOfReadings = numberOfReadings + 1
END METHOD

PUBLIC METHOD deleteAt(INTEGER indexToDelete)
    IF indexToDelete < 0 OR indexToDelete >= numberOfReadings THEN
        THROW IllegalArgumentException
    END IF

    FOR i = indexToDelete TO numberOfReadings - 2
        SET readings[i] = readings[i + 1]
    END FOR

    SET numberOfReadings = numberOfReadings - 1
    SET readings[numberOfReadings] = NULL
END METHOD

PUBLIC METHOD get(INTEGER indexToFetch) RETURNS SensorReading
    IF indexToFetch < 0 OR indexToFetch >= numberOfReadings THEN
        THROW IllegalArgumentException
    END IF

    RETURN readings[indexToFetch]
END METHOD

PUBLIC METHOD getCount() RETURNS INTEGER
    RETURN numberOfReadings
END METHOD

PUBLIC METHOD hasSensorID(String sensorIDToFind) RETURNS BOOLEAN
    DECLARE BOOLEAN sensorIDAlreadyExists = FALSE

    FOR i = 0 TO numberOfReadings - 1
        IF readings[i].getSensorID() EQUALS sensorIDToFind THEN
            SET sensorIDAlreadyExists = TRUE
        END IF
    END FOR

    RETURN sensorIDAlreadyExists
END METHOD

PUBLIC METHOD filterByZone(String zoneToMatch) RETURNS SensorArray
    DECLARE SensorArray matchingReadings = NEW SensorArray()

    FOR i = 0 TO numberOfReadings - 1
        IF readings[i].getZone() EQUALS zoneToMatch THEN
            CALL matchingReadings.add(readings[i])
        END IF
    END FOR

    RETURN matchingReadings
END METHOD

PUBLIC METHOD filterBySensorType(String typeToMatch) RETURNS SensorArray
```

```

    DECLARE SensorArray matchingReadings = NEW SensorArray()

    FOR i = 0 TO numberOfReadings - 1
        IF readings[i].getSensorType() EQUALS typeToMatch THEN
            CALL matchingReadings.add(readings[i])
        END IF
    END FOR

    RETURN matchingReadings
END METHOD

```

```

PRIVATE METHOD growArray()
    DECLARE INTEGER expandedCapacity = readings.length * CAPACITY_GROWTH_FACTOR
    DECLARE ARRAY OF SensorReading expandedReadings = NEW SensorReading ARRAY WITH SIZE
expandedCapacity

    FOR i = 0 TO numberOfReadings - 1
        SET expandedReadings[i] = readings[i]
    END FOR

    SET readings = expandedReadings
END METHOD

END CLASS

```

```

sensorReadings
=====

```

```

BEGIN PUBLIC CLASS SensorReading

    DECLARE PRIVATE STRING sensorID
    DECLARE PRIVATE STRING sensorType
    DECLARE PRIVATE STRING zone
    DECLARE PRIVATE DOUBLE value
    DECLARE PRIVATE Timestamp timestamp

    CONSTRUCTOR SensorReading()
        SET sensorID = ""
        SET sensorType = ""
        SET zone = ""
        SET value = 0.0
        SET timestamp = NEW Timestamp()
    END CONSTRUCTOR

    CONSTRUCTOR SensorReading(STRING sensorID, STRING sensorType, STRING zone, DOUBLE value,
Timestamp timestamp)
        SET this.sensorID = sensorID
        SET this.sensorType = sensorType
        SET this.zone = zone
        SET this.value = value
        SET this.timestamp = timestamp
    END CONSTRUCTOR

    CONSTRUCTOR SensorReading(SensorReading sourceReading)
        SET this.sensorID = sourceReading.sensorID
        SET this.sensorType = sourceReading.sensorType

```

```
    SET this.zone = sourceReading.zone
    SET this.value = sourceReading.value
    SET this.timestamp = NEW Timestamp(sourceReading.timestamp)
END CONSTRUCTOR

PUBLIC METHOD getSensorID() RETURNS STRING
    RETURN sensorID
END METHOD

PUBLIC METHOD getSensorType() RETURNS STRING
    RETURN sensorType
END METHOD

PUBLIC METHOD getZone() RETURNS STRING
    RETURN zone
END METHOD

PUBLIC METHOD getValue() RETURNS DOUBLE
    RETURN value
END METHOD

PUBLIC METHOD getTimestamp() RETURNS Timestamp
    RETURN timestamp
END METHOD

PUBLIC METHOD setSensorID(STRING sensorID)
    SET this.sensorID = sensorID
END METHOD

PUBLIC METHOD setSensorType(STRING sensorType)
    SET this.sensorType = sensorType
END METHOD

PUBLIC METHOD setZone(STRING zone)
    SET this.zone = zone
END METHOD

PUBLIC METHOD setValue(DOUBLE value)
    SET this.value = value
END METHOD

PUBLIC METHOD setTimestamp(Timestamp timestamp)
    SET this.timestamp = timestamp
END METHOD

PRIVATE STATIC METHOD getSafeRange(STRING sensorTypeToLookUp) RETURNS ARRAY OF DOUBLE
    DECLARE ARRAY OF DOUBLE safeRangeBounds = NULL

    IF sensorTypeToLookUp EQUALS "temperature" THEN
        SET safeRangeBounds = {18.0, 30.0}
    ELSE IF sensorTypeToLookUp EQUALS "humidity" THEN
        SET safeRangeBounds = {40.0, 70.0}
    ELSE IF sensorTypeToLookUp EQUALS "soilMoisture" THEN
        SET safeRangeBounds = {30.0, 60.0}
    ELSE IF sensorTypeToLookUp EQUALS "light" THEN
        SET safeRangeBounds = {300.0, 1200.0}
    END IF
```

```

    RETURN safeRangeBounds
END METHOD

```

```

PUBLIC METHOD isOutOfRange() RETURNS BOOLEAN
    DECLARE BOOLEAN readingIsOutOfRange = FALSE
    DECLARE ARRAY OF DOUBLE safeRangeBounds = CALL getSafeRange(sensorType)

    IF safeRangeBounds IS NULL THEN
        SET readingIsOutOfRange = FALSE
    ELSE IF value < safeRangeBounds[0] THEN
        SET readingIsOutOfRange = TRUE
    ELSE IF value > safeRangeBounds[1] THEN
        SET readingIsOutOfRange = TRUE
    ELSE
        SET readingIsOutOfRange = FALSE
    END IF

    RETURN readingIsOutOfRange
END METHOD

```

```

PUBLIC METHOD toString() RETURNS STRING
    DECLARE STRING timestampPrefix = "[" + timestamp.toString() + "]" "
    DECLARE STRING readingDetails = sensorID + " (" + sensorType + ", " + zone + "): " +
value
    RETURN timestampPrefix + readingDetails
END METHOD

```

```

END CLASS

```

```

Timestamp
=====

```

```

BEGIN PUBLIC CLASS Timestamp

```

```

    DECLARE PRIVATE INTEGER dayOfMonth
    DECLARE PRIVATE INTEGER monthOfYear
    DECLARE PRIVATE INTEGER year
    DECLARE PRIVATE INTEGER hour
    DECLARE PRIVATE INTEGER minute

```

```

    CONSTRUCTOR Timestamp()
        SET dayOfMonth = 0
        SET monthOfYear = 0
        SET year = 0
        SET hour = 0
        SET minute = 0
    END CONSTRUCTOR

```

```

    CONSTRUCTOR Timestamp(INTEGER dayOfMonth, INTEGER monthOfYear, INTEGER year, INTEGER
hour, INTEGER minute)
        SET this.dayOfMonth = dayOfMonth
        SET this.monthOfYear = monthOfYear
        SET this.year = year
        SET this.hour = hour
        SET this.minute = minute
    END CONSTRUCTOR

```

```

CONSTRUCTOR Timestamp(Timestamp sourceTimestamp)
    SET this.dayOfMonth = sourceTimestamp.dayOfMonth
    SET this.monthOfYear = sourceTimestamp.monthOfYear
    SET this.year = sourceTimestamp.year
    SET this.hour = sourceTimestamp.hour
    SET this.minute = sourceTimestamp.minute
END CONSTRUCTOR

```

```

PUBLIC METHOD getDayOfMonth() RETURNS INTEGER
    RETURN dayOfMonth
END METHOD

```

```

PUBLIC METHOD getMonthOfYear() RETURNS INTEGER
    RETURN monthOfYear
END METHOD

```

```

PUBLIC METHOD getYear() RETURNS INTEGER
    RETURN year
END METHOD

```

```

PUBLIC METHOD getHour() RETURNS INTEGER
    RETURN hour
END METHOD

```

```

PUBLIC METHOD getMinute() RETURNS INTEGER
    RETURN minute
END METHOD

```

```

PUBLIC METHOD setDayOfMonth(INTEGER dayOfMonth)
    SET this.dayOfMonth = dayOfMonth
END METHOD

```

```

PUBLIC METHOD setMonthOfYear(INTEGER monthOfYear)
    SET this.monthOfYear = monthOfYear
END METHOD

```

```

PUBLIC METHOD setYear(INTEGER year)
    SET this.year = year
END METHOD

```

```

PUBLIC METHOD setHour(INTEGER hour)
    SET this.hour = hour
END METHOD

```

```

PUBLIC METHOD setMinute(INTEGER minute)
    SET this.minute = minute
END METHOD

```

```

PUBLIC METHOD toString() RETURNS STRING
    DECLARE STRING dayPadded = CALL padWithLeadingZero(dayOfMonth)
    DECLARE STRING monthPadded = CALL padWithLeadingZero(monthOfYear)
    DECLARE STRING hourPadded = CALL padWithLeadingZero(hour)
    DECLARE STRING minutePadded = CALL padWithLeadingZero(minute)

```

```

    RETURN dayPadded + "/" + monthPadded + "/" + year + " " + hourPadded + ":" +
minutePadded

```

END METHOD

```
PRIVATE METHOD padWithLeadingZero(INTEGER numberToPad) RETURNS STRING
  IF numberToPad < 10 THEN
    RETURN "0" + numberToPad
  ELSE
    RETURN "" + numberToPad
  END IF
END METHOD
```

END CLASS

Math
====

BEGIN PUBLIC CLASS Math

```
PUBLIC STATIC METHOD totalReadings(SensorArray readingsToUse) RETURNS INTEGER
  RETURN readingsToUse.getCount()
END METHOD
```

```
PUBLIC STATIC METHOD average(SensorArray readingsToUse) RETURNS DOUBLE
  DECLARE DOUBLE averageValue = 0.0
  DECLARE INTEGER numberOfReadings = readingsToUse.getCount()

  IF numberOfReadings EQUALS 0 THEN
    SET averageValue = 0.0
  ELSE
    DECLARE DOUBLE runningTotal = 0.0

    FOR i = 0 TO numberOfReadings - 1
      SET runningTotal = runningTotal + readingsToUse.get(i).getValue()
    END FOR

    SET averageValue = runningTotal / numberOfReadings
  END IF

  RETURN averageValue
END METHOD
```

```
PUBLIC STATIC METHOD minimum(SensorArray readingsToUse) RETURNS DOUBLE
  DECLARE DOUBLE minimumValue = 0.0
  DECLARE INTEGER numberOfReadings = readingsToUse.getCount()

  IF numberOfReadings EQUALS 0 THEN
    SET minimumValue = 0.0
  ELSE
    DECLARE DOUBLE smallestSoFar = readingsToUse.get(0).getValue()

    FOR i = 1 TO numberOfReadings - 1
      IF readingsToUse.get(i).getValue() < smallestSoFar THEN
        SET smallestSoFar = readingsToUse.get(i).getValue()
      END IF
    END FOR

    SET minimumValue = smallestSoFar
  END IF
```

```

    RETURN minimumValue
END METHOD

```

```

PUBLIC STATIC METHOD maximum(SensorArray readingsToUse) RETURNS DOUBLE
    DECLARE DOUBLE maximumValue = 0.0
    DECLARE INTEGER numberOfReadings = readingsToUse.getCount()

    IF numberOfReadings EQUALS 0 THEN
        SET maximumValue = 0.0
    ELSE
        DECLARE DOUBLE largestSoFar = readingsToUse.get(0).getValue()

        FOR i = 1 TO numberOfReadings - 1
            IF readingsToUse.get(i).getValue() > largestSoFar THEN
                SET largestSoFar = readingsToUse.get(i).getValue()
            END IF
        END FOR

        SET maximumValue = largestSoFar
    END IF

    RETURN maximumValue
END METHOD

```

```

PUBLIC STATIC METHOD outOfRangeCount(SensorArray readingsToUse) RETURNS INTEGER
    DECLARE INTEGER numberOutOfRange = 0

    FOR i = 0 TO readingsToUse.getCount() - 1
        IF readingsToUse.get(i).isOutOfRange() EQUALS TRUE THEN
            SET numberOutOfRange = numberOutOfRange + 1
        END IF
    END FOR

    RETURN numberOutOfRange
END METHOD

```

```

PUBLIC STATIC METHOD outOfRangePercent(SensorArray readingsToUse) RETURNS DOUBLE
    DECLARE DOUBLE percentOutOfRange = 0.0
    DECLARE INTEGER numberOfReadings = readingsToUse.getCount()

    IF numberOfReadings EQUALS 0 THEN
        SET percentOutOfRange = 0.0
    ELSE
        DECLARE INTEGER numberOutOfRange = CALL outOfRangeCount(readingsToUse)
        SET percentOutOfRange = (numberOutOfRange AS DOUBLE / numberOfReadings) * 100
    END IF

    RETURN percentOutOfRange
END METHOD

```

```

END CLASS

```

```

Validator
=====

```

```

BEGIN PUBLIC CLASS Validator

```

```
DECLARE PRIVATE STATIC FINAL STRING ARRAY VALID_SENSOR_TYPES =
    {"temperature", "humidity", "soilMoisture", "light"}

DECLARE PRIVATE STATIC FINAL INTEGER MIN_DAY = 1
DECLARE PRIVATE STATIC FINAL INTEGER MAX_DAY = 31
DECLARE PRIVATE STATIC FINAL INTEGER MIN_MONTH = 1
DECLARE PRIVATE STATIC FINAL INTEGER MAX_MONTH = 12
DECLARE PRIVATE STATIC FINAL INTEGER MIN_YEAR = 2000
DECLARE PRIVATE STATIC FINAL INTEGER MAX_YEAR = 2100
DECLARE PRIVATE STATIC FINAL INTEGER MIN_HOUR = 0
DECLARE PRIVATE STATIC FINAL INTEGER MAX_HOUR = 23
DECLARE PRIVATE STATIC FINAL INTEGER MIN_MINUTE = 0
DECLARE PRIVATE STATIC FINAL INTEGER MAX_MINUTE = 59

PUBLIC STATIC METHOD isValidSensorType(String sensorTypeToValidate) RETURNS BOOLEAN
    IF sensorTypeToValidate IS NULL THEN
        RETURN FALSE
    END IF

    FOR i = 0 TO VALID_SENSOR_TYPES.length - 1
        IF VALID_SENSOR_TYPES[i] EQUALS sensorTypeToValidate THEN
            RETURN TRUE
        END IF
    END FOR

    RETURN FALSE
END METHOD

PUBLIC STATIC METHOD isValidZone(String zoneToValidate) RETURNS BOOLEAN
    RETURN zoneToValidate IS NOT NULL AND zoneToValidate.trim().length() > 0
END METHOD

PUBLIC STATIC METHOD isValidSensorID(String sensorIdToValidate) RETURNS BOOLEAN
    RETURN sensorIdToValidate IS NOT NULL AND sensorIdToValidate.trim().length() > 0
END METHOD

PUBLIC STATIC METHOD isValidDay(Integer dayToValidate) RETURNS BOOLEAN
    RETURN dayToValidate >= MIN_DAY AND dayToValidate <= MAX_DAY
END METHOD

PUBLIC STATIC METHOD isValidMonth(Integer monthToValidate) RETURNS BOOLEAN
    RETURN monthToValidate >= MIN_MONTH AND monthToValidate <= MAX_MONTH
END METHOD

PUBLIC STATIC METHOD isValidYear(Integer yearToValidate) RETURNS BOOLEAN
    RETURN yearToValidate >= MIN_YEAR AND yearToValidate <= MAX_YEAR
END METHOD

PUBLIC STATIC METHOD isValidHour(Integer hourToValidate) RETURNS BOOLEAN
    RETURN hourToValidate >= MIN_HOUR AND hourToValidate <= MAX_HOUR
END METHOD
```



```

PUBLIC STATIC METHOD isValidMinute(INTEGER minuteToValidate) RETURNS BOOLEAN
    RETURN minuteToValidate >= MIN_MINUTE AND minuteToValidate <= MAX_MINUTE
END METHOD

```

```

PUBLIC STATIC METHOD isNumeric(String stringToCheck) RETURNS BOOLEAN
    IF stringToCheck IS NULL OR stringToCheck.trim().length() EQUALS 0 THEN
        RETURN FALSE
    END IF

    TRY
        PARSE stringToCheck.trim() AS DOUBLE
        RETURN TRUE
    CATCH NumberFormatException
        RETURN FALSE
    END TRY
END METHOD

```

```

PUBLIC STATIC METHOD isInteger(String stringToCheck) RETURNS BOOLEAN
    IF stringToCheck IS NULL OR stringToCheck.trim().length() EQUALS 0 THEN
        RETURN FALSE
    END IF

    TRY
        PARSE stringToCheck.trim() AS INTEGER
        RETURN TRUE
    CATCH NumberFormatException
        RETURN FALSE
    END TRY
END METHOD

```

```

END CLASS

```

```

Logger
=====

```

```

BEGIN PUBLIC CLASS Logger

```

```

    DECLARE PRIVATE STATIC FINAL STRING LOG_FILE_NAME = "logged_data"
    DECLARE PRIVATE STATIC BufferedWriter logFileWriter = NULL

```

```

    PUBLIC STATIC METHOD open()
        TRY
            SET logFileWriter = OPEN LOG_FILE_NAME FOR APPENDING
            WRITE "starting session..." TO logFileWriter
            WRITE NEW LINE TO logFileWriter
            FLUSH logFileWriter
        CATCH Exception
            PRINT "could not open log file..."
            SET logFileWriter = NULL
        END TRY
    END METHOD

```

```

    PUBLIC STATIC METHOD log(String messageToLog)
        PRINT messageToLog

        IF logFileWriter IS NOT NULL THEN

```

```
        TRY
            WRITE messageToLog TO logFileWriter
            WRITE NEW LINE TO logFileWriter
            FLUSH logFileWriter
        CATCH IOException
            DO NOTHING
        END TRY
    END IF
END METHOD

PUBLIC STATIC METHOD logBlank()
    CALL log("")
END METHOD

PUBLIC STATIC METHOD close()
    IF logFileWriter IS NOT NULL THEN
        TRY
            WRITE "ending session" TO logFileWriter
            WRITE NEW LINE TO logFileWriter
            WRITE NEW LINE TO logFileWriter
            FLUSH logFileWriter
            CLOSE logFileWriter
        CATCH IOException
            DO NOTHING
        END TRY
    END IF
END METHOD

END CLASS
```